

UNIDAD DIDÁCTICA 01

Introducción a PHP_

Fundamentos del desarrollo web del lado del
servidor

Sintaxis

Variables y datos

Control de flujo

Formularios

APUNTES DE CLASE

Teoría · ejemplos de código · tablas de consulta

{ PHP }

GUÍA DE CONTENIDOS

Índice de la unidad

Siete bloques de aprendizaje, desde los fundamentos del lenguaje hasta el procesamiento y la validación de formularios.

01 ¿Qué es PHP?

- 1.1 Ejecución del código PHP
- 1.2 Lenguaje del Lado del Servidor
- 1.3 Integración con HTML y Scripting Embebido
- 1.4 Multiplataforma y Compatibilidad con Servidores
- 1.5 Código Abierto y Extensible
- 1.6 Facilidad de Aprendizaje y Amplia Comunidad
- 1.7 Escalabilidad y Performance
- 1.8 Seguridad
- 1.9 Aplicaciones Comunes de PHP

02 Instalación y Configuración de PHP

- 2.1 Paquetes de Entornos de Desarrollo Local
- 2.2 Instalación de XAMPP: Guía Paso a Paso

03 Sintaxis Básica de PHP

- 3.1 Inclusión de PHP en HTML
- 3.2 Estructura del Código PHP
- 3.3 Comentarios en PHP
- 3.4 Variables en PHP
- 3.5 Buenas Prácticas para Escribir Código PHP

04 Variables y Tipos de Datos en PHP

- 4.1 Declaración de Variables en PHP
- 4.2 Tipado Débil en PHP
- 4.3 Tipos de Datos en PHP
 - 4.3.2 Arrays y Maps: Funciones Clave y Ejemplos
- 4.4 Conversión de Tipos en PHP
- 4.5 Variables Predefinidas en PHP

05 Operadores y Expresiones en PHP

- 5.1 Operadores Aritméticos
- 5.2 Operadores de Asignación
- 5.3 Operadores de Comparación
- 5.4 Operadores Lógicos
- 5.5 Operadores de Incremento y Decremento

06 Estructuras de Control en PHP

- 6.1 Estructuras Condicionales
- 6.2 Estructuras de Repetición (Bucles)
- 6.3 Otras Estructuras de Control

07 Trabajo con Formularios en PHP

- 7.1 Métodos de Envío de Formularios: GET vs POST
- 7.2 Procesamiento de Formularios en PHP
- 7.3 Validación de Formularios en PHP: Un Enfoque Integral
 - 7.3.3 Consideraciones Adicionales: Mejores Prácticas para la Validación en PHP con Ejemplos
- 7.4 Subida de Archivos en Formularios

AMPLIACIONES PARA LOS EJERCICIOS

- | | |
|--------------------|--|
| 1.2.1-1.2.2 | Cliente, servidor y ejecución PHP en XAMPP. |
| 4.6 | Funciones de arrays para tickets, horarios y tests. |
| 6.4-6.6 | Funciones propias, archivos y utilidades auxiliares. |
| 7.1.3-7.1.4 | Select, radio, checkbox y campos ocultos. |
| 7.5-7.6 | Validación completa y corrección de tests. |

BLOQUE FORMATIVO

1. ¿Qué es PHP?

PHP, acrónimo de "**Hypertext Preprocessor**", PHP es un lenguaje de programación **de tipado dinámico**, cuyo código se ejecuta mediante el motor de PHP, y enfocado principalmente en el desarrollo del lado del servidor para aplicaciones web. Creado originalmente en 1994 por **Rasmus Lerdorf**, PHP ha evolucionado desde un conjunto de scripts de utilidad personal a un lenguaje completamente maduro y ampliamente utilizado, diseñado para generar contenido web dinámico, gestionar bases de datos y desarrollar aplicaciones robustas en múltiples entornos.

PHP es uno de los lenguajes más populares en el desarrollo web debido a su **facilidad de uso**, **flexibilidad** y la extensa comunidad que lo respalda. Su integración directa con HTML y su capacidad para interactuar con una variedad de bases de datos lo hacen ideal para aplicaciones web que requieren generación dinámica de contenido.

1.1 Ejecución del código PHP

PHP se ejecuta mediante un motor, habitualmente **Zend Engine**. Antes de ejecutar un script, el motor traduce el código fuente a instrucciones internas (*opcodes*), que después ejecuta. Normalmente no es necesario generar un ejecutable independiente como paso previo. El programa puede producir **HTML**, **JSON** u otros tipos de respuesta.

Este modelo permite modificar y ejecutar el código de manera ágil. Para mejorar el rendimiento, mecanismos como **OPcache** pueden conservar los *opcodes* ya generados y evitar repetir parte del procesamiento en cada solicitud.

1.2 Lenguaje del Lado del Servidor

PHP es un lenguaje de programación **del lado del servidor**. Esto significa que el código PHP se ejecuta en el servidor web (en lugar del navegador del cliente) y el resultado procesado se envía al cliente en forma de HTML. **El cliente (generalmente un navegador) no ve el código PHP, sino únicamente la salida final generada por él.**

Por ejemplo, cuando un cliente solicita una página web, el servidor ejecuta el script PHP, interactúa con bases de datos u otros servicios si es necesario, genera HTML y lo envía al navegador. Este enfoque permite la creación de sitios web **dinámicos**, en los que el contenido cambia en función de la interacción del usuario o las entradas del sistema, como en tiendas en línea, sistemas de gestión de contenido (CMS), foros, etc.

BLOQUE 01 · FUNDAMENTOS / 1.2.1

Cliente y servidor: quién hace qué

Un **cliente** es un programa que solicita un servicio o un recurso. Un **servidor** es un programa que recibe esa petición, la atiende y devuelve una respuesta.

En una web, el **navegador** (Chrome, Firefox, etc.) actúa como cliente; el **servidor web** atiende sus peticiones. Piensa en un restaurante: el cliente pide un plato y la cocina lo prepara. El cliente recibe el resultado sin tener que cocinarlo. De modo parecido, en una aplicación PHP el navegador pide una página y el servidor prepara su respuesta.

Un ejemplo cercano: consultar las notas

1. Abres Chrome y pulsas «Consultar mis notas» en la web del centro.
2. **Chrome envía una petición HTTP** al servidor de la aplicación.
3. **El servidor procesa la solicitud:** comprueba tu identidad, los permisos necesarios y consulta las notas en la base de datos.
4. El servidor **devuelve una respuesta** con la información que corresponda.
5. Chrome interpreta la respuesta y **muestra la página** en pantalla.

El navegador muestra la información y recoge tus acciones. El servidor puede consultar datos, aplicar reglas, comprobar permisos o guardar cambios. **Ambos envían y reciben información:** por ejemplo, el navegador manda los datos de un formulario y luego recibe el resultado del procesamiento.

«Servidor» puede significar dos cosas

Término	Qué significa	Ejemplo
Cliente	Programa que solicita un servicio.	Chrome al pedir una página.
Servidor (software)	Programa que atiende peticiones.	Apache, incluido en XAMPP.
Servidor (equipo)	Ordenador donde se ejecuta ese software.	El equipo que aloja la web del centro.

Idea clave: un servidor no tiene que ser una máquina enorme. Tu ordenador puede ejecutar programas que actúen como servidores mientras tu navegador hace de cliente.

BLOQUE 01 · FUNDAMENTOS / 1.2.2

PHP en el servidor: ejemplo con XAMPP

Al trabajar en local, **cliente y servidor pueden estar en el mismo equipo**. En el entorno que utilizaremos, cada componente tiene su cometido:

Componente	Función en esta práctica
Chrome	Cliente: solicita páginas y presenta el resultado.
Apache	Servidor web: recibe las solicitudes HTTP y coordina la respuesta.
PHP	Motor que ejecuta el código PHP cuando la petición lo requiere.
XAMPP	Paquete para instalar y poner en marcha estos componentes.

Cuando escribes `http://localhost/index.php`, `localhost` apunta a **tu propio ordenador**. Para esta prueba local no necesitas conexión a Internet; sí necesitas tener Apache iniciado y el archivo accesible desde `htdocs`.

¿Dónde se ejecuta el código?

Guarda este archivo como `index.php` dentro de la carpeta `htdocs` de XAMPP:

```
<h1>Mi primera página</h1>
<p>
  El resultado es:
  <?php echo 2 + 3; ?>
</p>
```

1. Chrome solicita `index.php` a Apache.
2. Apache gestiona la petición y permite que PHP procese el archivo.
3. PHP calcula `2 + 3` y genera el texto `5`.
4. El servidor devuelve al navegador el HTML resultante:

```
<h1>Mi primera página</h1>
<p>
  El resultado es:
  5
</p>
```

Chrome recibe el resultado, no la instrucción `echo 2 + 3`. En este uso habitual, el navegador interpreta HTML, aplica CSS y puede ejecutar JavaScript, pero no ejecuta PHP.

Para comprobarlo: accede mediante `http://localhost/index.php` con Apache iniciado. Si abres `index.php` con doble clic (`file:///...`), no pasa por el servidor y el PHP no se ejecuta.

1.3 Integración con HTML y Scripting Embebido

PHP es conocido por su capacidad para integrarse de forma nativa con **HTML**, lo que permite a los desarrolladores intercalar código PHP con HTML en el mismo archivo. Esto significa que los desarrolladores pueden combinar fácilmente lógica de programación con el marcado HTML para generar páginas web dinámicas sin necesidad de separar la lógica de la presentación, como se haría en lenguajes basados en plantillas más rígidos.

Ejemplo básico de código PHP embebido en HTML:

• PHP

```
<html>
  <body>
    <h1>Bienvenido a mi sitio web</h1>
    <?php
      echo "Hoy es " . date("Y/m/d") . "<br>";
      echo "La hora actual es " . date("h:i:sa");
    ?>
  </body>
</html>
```

En este ejemplo, el código PHP genera la fecha y hora actuales en un documento HTML. Esta capacidad de integrar directamente PHP en un archivo HTML lo hace sumamente flexible para generar contenido dinámico en la web.

1.4 Multiplataforma y Compatibilidad con Servidores

PHP es **multiplataforma**, lo que significa que puede ejecutarse en una amplia variedad de sistemas operativos, como **Linux**, **Windows**, **macOS** y entornos UNIX. Además, es compatible con los principales servidores web, como **Apache**, **Nginx**, e **IIS** (Internet Information Services). Esta flexibilidad permite que los desarrolladores puedan elegir la plataforma que mejor se ajuste a sus necesidades o a la infraestructura existente de su aplicación.

El soporte multiplataforma no sólo abarca sistemas operativos y servidores, sino que también incluye **compatibilidad con múltiples bases de datos**. PHP se integra nativamente con sistemas de bases de datos relacionales como **MySQL**, **PostgreSQL**, **Microsoft SQL Server**, y **Oracle**. Además, existen extensiones para interactuar con bases de datos no relacionales (NoSQL) como **MongoDB**, lo que lo hace adecuado para proyectos de todo tipo.

1.5 Código Abierto y Extensible

PHP es un lenguaje **open-source**, lo que significa que su código fuente es accesible y está disponible gratuitamente para cualquier persona. Esto fomenta la colaboración, el desarrollo y la mejora constante por parte de la comunidad global de programadores. La naturaleza abierta de PHP permite a los desarrolladores modificar el lenguaje y su comportamiento, adaptándolo a sus necesidades específicas cuando es necesario.

Además de su núcleo, PHP cuenta con una amplia variedad de **extensiones** (módulos) que añaden funcionalidades adicionales, como el manejo de gráficos, la interacción con protocolos de red, la criptografía, entre otros. Algunos de los módulos más utilizados incluyen:

- **GD**: Para la manipulación de imágenes.

- **cURL**: Para la transferencia de datos mediante URLs.
- **PDO** (PHP Data Objects): Para la abstracción de bases de datos.

1.6 Facilidad de Aprendizaje y Amplia Comunidad

PHP es conocido por ser uno de los lenguajes más **accesibles** para desarrolladores principiantes. Su sintaxis es relativamente sencilla y no requiere una configuración compleja para empezar a desarrollar aplicaciones básicas. Esta facilidad de uso, combinada con su versatilidad, lo convierte en una excelente opción tanto para quienes están comenzando en el desarrollo web como para desarrolladores experimentados que buscan un lenguaje potente para crear aplicaciones web complejas.

La popularidad de PHP también ha dado lugar a una **comunidad extensa y activa**, que proporciona una gran cantidad de recursos, desde documentación oficial y tutoriales, hasta librerías de terceros y frameworks que agilizan el desarrollo. Algunos de los frameworks más populares basados en PHP incluyen **Laravel**, **Symfony**, y **CodeIgniter**, que proporcionan estructuras sólidas para desarrollar aplicaciones web a gran escala.

1.7 Escalabilidad y Performance

Aunque PHP es conocido por su facilidad de uso, también es un lenguaje potente y escalable para aplicaciones de gran tamaño. Con el uso adecuado de herramientas como **OPcache** (que cachea el código PHP precompilado para mejorar el rendimiento), junto con el enfoque en la optimización del código y el uso de frameworks robustos, PHP puede manejar aplicaciones web con alto tráfico sin perder eficiencia.

El lenguaje también soporta **programación orientada a objetos (POO)**, lo que facilita la creación de aplicaciones más organizadas, reutilizables y mantenibles. Este paradigma es crucial en proyectos de gran envergadura donde se requiere una estructura de código modular y escalable.

1.8 Seguridad

La seguridad de una aplicación PHP depende del diseño, la configuración y las prácticas de desarrollo. Como en cualquier tecnología web, pueden aparecer **vulnerabilidades** si no se aplican las medidas adecuadas. Entre los riesgos comunes se incluyen **inyección SQL**, **cross-site scripting (XSS)**, y **cross-site request forgery (CSRF)**. Sin embargo, PHP incluye herramientas integradas y buenas prácticas para mitigar estos riesgos, como:

- **Preparación de consultas** (prepared statements) para evitar la inyección SQL.
- **Escape contextual de los datos al mostrarlos en HTML** para mitigar XSS; la validación de entrada cumple una función complementaria.
- **Tokens CSRF** para protegerse contra ataques de falsificación de solicitudes.

Además, existen extensiones como **Sodium** para criptografía avanzada y **OpenSSL** para garantizar la transmisión segura de datos.

1.9 Aplicaciones Comunes de PHP

PHP se utiliza en una amplia gama de aplicaciones web, desde pequeñas páginas informativas hasta grandes plataformas web como **Facebook** (en sus inicios), **Wikipedia**, y **WordPress**. Las áreas comunes de uso de PHP incluyen:

- **Sistemas de gestión de contenido (CMS):** WordPress, Joomla y Drupal están contruidos sobre PHP.
- **E-commerce:** Plataformas como Magento y OpenCart están desarrolladas en PHP.
- **Foros y redes sociales:** Aplicaciones como phpBB y MediaWiki (Wikipedia) están escritas en PHP.
- **APIs y servicios web:** PHP se utiliza para crear APIs RESTful y SOAP que interactúan con aplicaciones móviles y otros servicios.

BLOQUE FORMATIVO

2. Instalación y Configuración de PHP

Para comenzar a desarrollar aplicaciones con PHP, es necesario configurar un entorno de desarrollo local que permita ejecutar scripts PHP en un servidor web y, opcionalmente, interactuar con bases de datos. Aunque PHP se puede instalar de forma independiente, es habitual utilizar paquetes preconfigurados que incluyen todo lo necesario para el desarrollo web. Estos paquetes son muy útiles para evitar la configuración manual de cada componente y simplificar la instalación.

2.1 Paquetes de Entornos de Desarrollo Local

Los paquetes de entornos de desarrollo local más comunes son:

- **XAMPP:** Un entorno de servidor multiplataforma que incluye Apache (servidor web), MySQL (en realidad MariaDB) y PHP. Está disponible para Windows, macOS, y Linux, lo que lo convierte en una opción versátil.
- **WAMP:** Un paquete similar a XAMPP, pero diseñado exclusivamente para el sistema operativo Windows. WAMP incluye Apache, MySQL, y PHP. Es ideal para desarrolladores que trabajan en Windows y no necesitan soporte multiplataforma.
- **MAMP:** Diseñado específicamente para usuarios de macOS, MAMP es un paquete que incluye Apache, MySQL, y PHP. También está disponible una versión para Windows, aunque su popularidad es mayor en macOS.

Estos entornos facilitan el desarrollo de aplicaciones PHP sin necesidad de instalar y configurar manualmente cada componente de un servidor web y base de datos. Además, incluyen interfaces gráficas que permiten gestionar fácilmente los servicios asociados (servidor web, bases de datos, etc.).

2.2 Instalación de XAMPP: Guía Paso a Paso

XAMPP es uno de los entornos de desarrollo más populares debido a su soporte multiplataforma y su facilidad de uso. A continuación, se detallan los pasos para instalar y configurar XAMPP en un sistema operativo Windows, aunque el proceso es similar para macOS y Linux.

Paso 1: Descargar XAMPP

- Accede al sitio oficial de XAMPP en [apachefriends.org](https://www.apachefriends.org).
- Selecciona la versión de XAMPP correspondiente a tu sistema operativo (Windows, macOS o Linux). Las versiones de XAMPP vienen con diferentes versiones de PHP, por lo que puedes seleccionar la más adecuada para tu proyecto (por ejemplo, si necesitas PHP 7.4 o PHP 8.x).

Paso 2: Ejecutar el Instalador

- Una vez descargado el archivo instalador, ejecuta el instalador de XAMPP.

BLOQUE 02 · ENTORNO DE DESARROLLO

2.2 Instalación de XAMPP · puesta en marcha

Después de descargar XAMPP desde su web oficial y ejecutar el instalador, utiliza el panel de control para iniciar los componentes que necesites. La instalación local será nuestro entorno de aprendizaje.

Paso 2. Instalar los componentes necesarios

Durante la instalación, comprueba que se incluyen **Apache, PHP, MariaDB (mostrado como «MySQL» en el panel de XAMPP) y phpMyAdmin**. No necesitas instalar herramientas adicionales para empezar con PHP y bases de datos.

En Windows: elige una carpeta de instalación con permisos adecuados y sigue los avisos del asistente. No es necesario conceder permisos de administrador a cualquier aplicación para usar PHP.

Paso 3. Iniciar el servidor

1. Abre el **Panel de control de XAMPP**.
2. Pulsa **Start** en la fila de **Apache** para servir páginas PHP.
3. Cuando vayas a usar bases de datos, inicia también el servicio **MySQL**, que en XAMPP suele corresponder a MariaDB.

Si Apache no arranca, comprueba si el puerto 80 está ocupado por otro programa. Si la instalación usa otro puerto (por ejemplo, 8080), la dirección será `http://localhost:8080/`.

Paso 4. Crear el primer archivo PHP

En Windows, la carpeta de publicación predeterminada suele ser `C:\xampp\htdocs`. Crea allí un archivo llamado `index.php`:

```
<?php
    echo "¡Hola, mundo!";
?>
```

Con Apache iniciado, abre `http://localhost/index.php` en el navegador. Deberás ver **¡Hola, mundo!** como resultado de la ejecución de PHP.

Para mantener ordenados los ejercicios, puedes crear una subcarpeta distinta dentro de `htdocs` para cada práctica: `htdocs/UD1/01-primer-formulario/`, por ejemplo.

BLOQUE 02 · BASES DE DATOS

2.2 Gestión inicial de bases de datos

Paso 5. Trabajar primero con phpMyAdmin

En las primeras prácticas de bases de datos usaremos **phpMyAdmin**, una herramienta web que se abre en el navegador y permite crear bases de datos, tablas, consultar registros y ejecutar instrucciones SQL. Está disponible habitualmente con XAMPP.

1. En el panel de XAMPP inicia **Apache** y **MySQL** (MariaDB).
2. Abre <http://localhost/phpmyadmin/> en el navegador.
3. En la interfaz de phpMyAdmin, selecciona **Nueva**, escribe el nombre de la base de datos y pulsa **Crear**.
4. Cuando se estudien las consultas SQL, podrás utilizar la pestaña **SQL** para ejecutarlas.

Importante: para las primeras prácticas de esta unidad solo necesitas Apache y PHP. El servicio de base de datos se inicia cuando trabajas con datos; phpMyAdmin es un administrador, no el motor de base de datos.

MySQL Workbench: alternativa opcional

No es necesario instalar ni utilizar MySQL Workbench para seguir las primeras clases. Se puede emplear voluntariamente más adelante como programa de escritorio para modelar esquemas y ejecutar consultas.

Si decides probarlo, instala una versión compatible con tu sistema, inicia el servicio de base de datos y configura una conexión a `localhost`, con el puerto y las credenciales efectivamente establecidos en tu instalación.

Recuerda que **XAMPP suele distribuir MariaDB**; ciertas funciones específicas de MySQL Workbench pueden no ser compatibles con ese servidor. Por eso usaremos **phpMyAdmin como herramienta inicial de referencia**.

Comprueba que tu entorno funciona

Acción	Qué debes comprobar
http://localhost/	Apache responde y muestra una página local.
http://localhost/index.php	El archivo PHP genera la salida esperada.
http://localhost/phpmyadmin/	Se abre phpMyAdmin con Apache y MariaDB iniciados.

BLOQUE FORMATIVO

3. Sintaxis Básica de PHP

PHP es un lenguaje de programación que se integra directamente con HTML, permitiendo que las páginas web dinámicas se generen y rendericen en el servidor antes de enviarse al navegador del cliente. La sintaxis de PHP es sencilla y flexible, y su capacidad para integrarse con HTML de forma transparente lo hace una opción popular para el desarrollo web. A continuación, se presentan los elementos esenciales de la sintaxis básica de PHP y cómo utilizarlo de manera efectiva en archivos web.

3.1 Inclusión de PHP en HTML

PHP está diseñado para ser embebido directamente dentro de documentos HTML. Para indicarle al servidor que procese un bloque de código PHP, debes envolver el código PHP entre las etiquetas de apertura `<?php` y cierre `?>`. Cualquier código que se coloque dentro de estas etiquetas será interpretado por el servidor, mientras que todo lo que esté fuera de estas etiquetas será tratado como HTML estándar.

Ejemplo de PHP embebido en un archivo HTML:

• PHP

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi primera página PHP</title>
</head>
<body>
  <h1>¡Hola, mundo!</h1>
  <?php
    echo "Este es un mensaje generado con PHP.";
  ?>
</body>
</html>
```

En este ejemplo, el navegador recibirá la salida como HTML. La etiqueta `<?php ... ?>` indica al servidor que procese el código PHP y lo convierta en HTML antes de enviarlo al navegador. El resultado sería la página HTML que incluye el texto "Este es un mensaje generado con PHP." donde se encuentra el bloque `echo`.

Importancia de las Etiquetas PHP

Las etiquetas de PHP (`<?php` y `?>`) son esenciales para distinguir el código PHP del HTML. Si omites estas etiquetas, el servidor no sabrá que debe procesar el código PHP y lo tratará como texto plano. Esto también significa que puedes mezclar PHP y HTML sin problemas dentro del mismo archivo, permitiendo la generación dinámica de contenido web.

3.2 Estructura del Código PHP

La estructura básica de un script PHP sigue convenciones comunes en otros lenguajes de programación, pero con sus propias peculiaridades. A continuación se describen algunos de los elementos más importantes.

3.2.1 Sentencias y Bloques de Código

PHP utiliza el **punto y coma** (`;`) para finalizar la mayoría de las sentencias, como asignaciones y llamadas a `echo`. Los bloques delimitados por llaves (`{}`), por ejemplo en `if` o `for`, no necesitan un punto y coma detrás de su llave de cierre.

Ejemplo:

• PHP

```
<?php
    echo "Hola, mundo!"; // Punto y coma para terminar la sentencia
    $nombre = "Juan"; // Asignación de valor a una variable
?>
```

3.2.2 Función `echo` y Generación de Salida

`echo` es una construcción del lenguaje PHP que se utiliza para generar salida. Generalmente, se utiliza para imprimir texto, variables o resultados en la página HTML que será enviada al navegador.

Ejemplo de uso de `echo`:

• PHP

```
<?php
    echo "Este es un mensaje generado con PHP."; // Imprime texto
    echo "<br>"; // Imprime una etiqueta HTML de salto de línea
    $mensaje = "Bienvenidos a mi sitio web";
    echo $mensaje; // Imprime el contenido de la variable $mensaje
?>
```

En este ejemplo, `echo` imprime directamente el texto y también el valor almacenado en la variable `$mensaje`. Puede generar tanto texto plano como contenido HTML, lo que permite una integración fluida con las páginas web.

3.3 Comentarios en PHP

Los comentarios permiten documentar el código sin alterar su ejecución. PHP admite dos formas habituales de comentario de una línea (`//` y `#`) y una forma de comentario de varias líneas (`/* ... */`).

3.3.1 Comentario de una línea:

1. **Comentarios con `//`:** Este es el método más común para hacer comentarios de una sola línea. Todo lo que se escriba después de `//` en la misma línea será tratado como comentario.

Ejemplo:

• PHP

```
<?php
// Este es un comentario de una línea
echo "Hola, mundo!";
?>
```

2. **Comentarios con #:** Aunque menos común, PHP también permite usar `#` para comentarios de una sola línea, similar a cómo se hace en lenguajes como Bash.

Ejemplo:

• PHP

```
<?php
# Esto también es un comentario de una línea
echo "Este es otro ejemplo de comentario.";
?>
```

3.3.2 Comentario de varias líneas:

Cuando necesitas hacer anotaciones más extensas, puedes utilizar comentarios de varias líneas. Se utilizan las mismas convenciones que en lenguajes como C, C++ o Java: abrir el comentario con `/*` y cerrarlo con `*/`.

Ejemplo de comentario de varias líneas:

• PHP

```
<?php
/*
    Esto es un comentario
    de varias líneas.
    Puedes usarlo para escribir
    descripciones más largas sobre el código.
*/
echo "Comentarios de varias líneas son útiles para documentar scripts grandes.";
?>
```

Este tipo de comentario es ideal para descripciones detalladas, como explicaciones de la lógica del código o anotaciones sobre su funcionamiento.

3.4 Variables en PHP

En PHP, las variables se declaran con el prefijo `$` y no requieren que especifiques el tipo de dato. PHP es un lenguaje **de tipado dinámico**, lo que significa que una variable puede cambiar su tipo en tiempo de ejecución. Los nombres de las variables deben comenzar con una letra o guion bajo, y pueden contener letras, números o guiones bajos, pero no pueden comenzar con un número.

Ejemplo de declaración y uso de variables:

• PHP

```
<?php
    $nombre = "María"; // Variable de tipo cadena (string)
    $edad = 25; // Variable de tipo entero (int)
    $precio = 12.99; // Variable de tipo flotante (float)

    echo "Nombre: " . $nombre . "<br>";
    echo "Edad: " . $edad . "<br>";
    echo "Precio: $" . $precio . "<br>";
?>
```

En este ejemplo, vemos cómo PHP permite declarar y utilizar variables sin necesidad de definir explícitamente su tipo de dato. PHP detecta automáticamente el tipo basado en el valor asignado.

3.5 Buenas Prácticas para Escribir Código PHP

1. **Consistencia en el uso de etiquetas PHP:** Siempre usa la forma estándar de las etiquetas PHP (`<?php ... ?>`). Evita las etiquetas cortas para garantizar la portabilidad de tu código.
2. **Comentarios claros y descriptivos:** Utiliza comentarios para describir la lógica del código y explicar secciones complejas. Esto facilitará la colaboración con otros desarrolladores y el mantenimiento a largo plazo.
3. **Nombrado de variables descriptivas:** Asegúrate de usar nombres de variables descriptivos y consistentes para que el código sea más fácil de entender. En lugar de `$x`, utiliza nombres como `$precioProducto` o `$nombreUsuario`.
4. **Separación de lógica y presentación:** Aunque PHP permite mezclar HTML y código de manera sencilla, es buena práctica separar la lógica del negocio (PHP) de la presentación (HTML). Esto se puede lograr utilizando plantillas o frameworks que respeten el patrón MVC.

BLOQUE FORMATIVO

4. Variables y Tipos de Datos en PHP

En PHP, las **variables** son fundamentales para el desarrollo de aplicaciones, ya que permiten almacenar y manipular datos a lo largo de la ejecución de un script. PHP es un lenguaje **de tipado dinámico**: el tipo del valor de una variable se determina durante la ejecución, por lo que no necesitas declarar previamente el tipo de cada variable. Esta flexibilidad en la tipificación es una de las características que hacen de PHP un lenguaje accesible, pero también es importante comprender cómo maneja y trata los diferentes tipos de datos.

4.1 Declaración de Variables en PHP

Las variables en PHP se identifican mediante el símbolo `$` seguido de un nombre de variable. El nombre de la variable puede contener letras, números y guiones bajos, pero debe comenzar con una letra o un guion bajo.

Reglas para nombres de variables:

- Deben comenzar con una letra o un guion bajo (`_`).
- Pueden contener letras, números y guiones bajos, pero no otros caracteres especiales.
- PHP distingue entre mayúsculas y minúsculas, por lo que `$Variable` y `$variable` son dos variables distintas.

Ejemplo de declaración de variables en PHP:

```
• PHP

<?php
    $nombre = "Juan";           // Cadena de texto (string)
    $edad = 25;                 // Entero (integer)
    $altura = 1.75;             // Flotante (float)
    $esEstudiante = true;      // Booleano (boolean)

?>
```

En este ejemplo, `$nombre` almacena una cadena de texto, `$edad` es un entero, `$altura` es un número flotante y `$esEstudiante` es un booleano.

4.2 Tipado Débil en PHP

En PHP, no es necesario declarar el tipo de una variable. El lenguaje determina automáticamente el tipo de dato según el valor que se le asigne. Esta característica se denomina **tipado dinámico**. El concepto de tipado débil se refiere, por separado, a determinadas conversiones implícitas que PHP permite entre tipos. A diferencia de lenguajes de tipado estático, como Java o C++, donde el tipo de una variable debe definirse explícitamente, PHP permite que el tipo cambie en tiempo de ejecución si se le asigna un nuevo valor de un tipo diferente.

Ejemplo de tipado dinámico:

• PHP

```
<?php
    $variable = 5;           // Entero
    $variable = "Hola";     // Ahora es una cadena de texto
?>
```

En este caso, la variable `$variable` comienza almacenando un valor entero (`5`), pero luego cambia su tipo a una cadena de texto cuando se le asigna `"Hola"` .

4.3 Tipos de Datos en PHP

PHP soporta una variedad de tipos de datos nativos, que se pueden dividir en dos categorías principales: **tipos escalares** y **tipos compuestos**. Además, existe un tipo especial, **NULL**, que representa la ausencia de valor.

4.3.1 Tipos Escalares

Los tipos escalares son tipos de datos simples que contienen un solo valor. Incluyen los siguientes:

1. **Enteros (integer)**: Los enteros son números sin decimales. Pueden ser positivos o negativos. Los enteros en PHP pueden representarse en varias bases numéricas:
 - **Decimal** (base 10): Ejemplo: `25`
 - **Octal** (base 8): Ejemplo: `075` (equivale a `61` en decimal)
 - **Hexadecimal** (base 16): Ejemplo: `0x1A` (equivale a `26` en decimal)
 - **Binario** (base 2): Ejemplo: `0b1101` (equivale a `13` en decimal)

Ejemplo:

• PHP

```
<?php
    $edad = 30; // Entero
    $negativo = -15; // Entero negativo
?>
```

2. **Flotantes (float)**: Los flotantes, también llamados **dobles** o **puntos flotantes**, son números que contienen una parte decimal. Estos números se representan utilizando la notación estándar de punto decimal o la notación científica.

Ejemplo:

• PHP

```
<?php
    $precio = 9.99; // Flotante con decimal
    $cientifico = 3e2; // Equivale a 300 (3 * 10^2)
?>
```

3. **Cadenas de texto (string)**: Las cadenas de texto son secuencias de caracteres. Pueden ser delimitadas con comillas simples (`'`) o dobles (`"`), y cada una tiene su propio comportamiento. Las comillas dobles permiten interpolar variables y reconocer ciertas secuencias de escape (como `\n`); las comillas simples no interpolan variables y reconocen menos secuencias de escape.

Ejemplo:

• PHP

```
<?php
    $saludo = "Hola, $nombre"; // Comillas dobles permiten interpolación de variables
    $saludoSimple = 'Hola, Juan'; // Comillas simples, no interpolan
?>
```

4. **Booleanos (boolean):** Los booleanos representan dos estados: **true** (verdadero) o **false** (falso). Son típicamente usados en evaluaciones condicionales y lógica de control.

Ejemplo:

• PHP

```
<?php
    $esAdmin = true; // Valor booleano verdadero
    $esInvitado = false; // Valor booleano falso
?>
```

4.3.2 Arrays y Maps: Funciones Clave y Ejemplos

En PHP, los **arrays** son estructuras de datos fundamentales que permiten almacenar colecciones de valores bajo un mismo nombre. Son extremadamente flexibles y pueden ser utilizados tanto como **arrays indexados numéricamente** (donde las posiciones están definidas por números enteros) como **arrays asociativos (maps)**, que utilizan claves personalizadas como índices. Además, pueden contener datos de diversos tipos, incluyendo números, cadenas, booleanos e incluso otros arrays, lo que permite crear estructuras más complejas.

A continuación se explican los tipos de arrays más comunes y algunas de las funciones más útiles que se pueden utilizar con ellos:

Arrays Indexados Numéricamente

Los **arrays indexados numéricamente** son aquellos en los que cada elemento está asociado a un índice numérico, comenzando en 0. Son ideales para almacenar listas de elementos en un orden específico.

Ejemplo básico de array indexado:

• PHP

```
<?php
    $numeros = array(1, 2, 3, 4, 5);
    echo $numeros[0]; // Imprime 1
?>
```

Aquí, el array `$numeros` contiene cinco valores, donde el índice 0 almacena el valor 1, el índice 1 almacena el valor 2, y así sucesivamente.

Funciones más útiles para trabajar con arrays indexados:

1. **count()**: Devuelve el número de elementos en un array.

- PHP

```
echo count($numeros); // Imprime 5
```

2. **array_push()** : Añade uno o más elementos al final del array.

- PHP

```
array_push($numeros, 6, 7);  
print_r($numeros); // Imprime Array ( [0] => 1 [1] => 2 [2] => 3 [3] => 4 [4] => 5 [5] => 6 [6] => 7 )
```

3. **array_pop()** : Elimina y devuelve el último valor del array.

- PHP

```
$ultimo_numero = array_pop($numeros);  
echo $ultimo_numero; // Si se ejecuta tras array_push(), imprime 7
```

4. **sort()** : Ordena los elementos de un array en orden ascendente.

- PHP

```
sort($numeros);  
print_r($numeros); // Imprime Array ( [0] => 1 [1] => 2 [2] => 3 [3] => 4 [4] => 5 )
```

5. **array_merge()** : Combina dos o más arrays.

- PHP

```
$mas_numeros = array(6, 7, 8);  
$resultado = array_merge($numeros, $mas_numeros);  
print_r($resultado); // Imprime Array ( [0] => 1 [1] => 2 [2] => 3 [3] => 4 [4] => 5 [5] => 6 [6] => 7 [7] => 8 )
```

Arrays Asociativos (Maps)

Los **arrays asociativos**, también conocidos como **maps**, permiten utilizar claves personalizadas como índices en lugar de números. Esto es útil cuando se necesita asociar un valor con una clave descriptiva, como en el caso de almacenar información de una persona (nombre, edad, dirección, etc.).

Ejemplo básico de array asociativo:

- PHP

```
<?php  
$persona = array("nombre" => "Juan", "edad" => 25);  
echo $persona["nombre"]; // Imprime "Juan"  
?>
```

En este ejemplo, el array `$persona` asocia la clave `"nombre"` con el valor `"Juan"` y la clave `"edad"` con el valor `25`.

Funciones más útiles para trabajar con arrays asociativos:

1. **array_keys()** : Devuelve todas las claves de un array asociativo.

- PHP

```
$claves = array_keys($persona);  
print_r($claves); // Imprime Array ( [0] => nombre [1] => edad )
```

2. **array_values()** : Devuelve todos los valores de un array asociativo.

- PHP

```
$valores = array_values($persona);  
print_r($valores); // Imprime Array ( [0] => Juan [1] => 25 )
```

3. **isset()** : Comprueba si una clave está definida y su valor es distinto de **null**.

- PHP

```
if (isset($persona["nombre"])) {  
    echo "El nombre está definido."; // Imprime "El nombre está definido."  
}
```

4. **array_key_exists()** : Verifica si una clave específica existe en un array.

- PHP

```
if (array_key_exists("edad", $persona)) {  
    echo "La clave 'edad' existe en el array."; // Imprime "La clave 'edad' existe en el array."  
}
```

5. **unset()** : Elimina un elemento de un array asociativo mediante su clave.

- PHP

```
unset($persona["edad"]);  
print_r($persona); // Imprime Array ( [nombre] => Juan )
```

6. **array_combine()** : Combina dos arrays, uno de claves y otro de valores, en un solo array asociativo.

- PHP

```
$claves = array("nombre", "edad");  
$valores = array("Ana", 30);  
$persona = array_combine($claves, $valores);  
print_r($persona); // Imprime Array ( [nombre] => Ana [edad] => 30 )
```

Arrays Multidimensionales

Los arrays multidimensionales son arrays que contienen otros arrays como elementos, lo que permite estructurar datos en niveles más profundos.

Ejemplo de array multidimensional:

• PHP

```
<?php
$personas = array(
    array("nombre" => "Juan", "edad" => 25),
    array("nombre" => "Ana", "edad" => 30)
);
echo $personas[1]["nombre"]; // Imprime "Ana"
?>
```

Aquí, el array `$personas` contiene dos arrays, cada uno con claves `"nombre"` y `"edad"`. Se accede al nombre de la segunda persona (`Ana`) utilizando los índices correspondientes.

Funciones útiles para arrays multidimensionales:

1. **`array_map()`** : Aplica una función a cada elemento de un array (útil también para arrays multidimensionales).

• PHP

```
$nombres = array_map(function($persona) {
    return $persona["nombre"];
}, $personas);
print_r($nombres); // Imprime Array ( [0] => Juan [1] => Ana )
```

2. **`array_walk_recursive()`** : Aplica una función a cada elemento de un array de forma recursiva.

• PHP

```
array_walk_recursive($personas, function($valor, $clave) {
    echo "$clave: $valor\n";
});
// Imprime "nombre: Juan", "edad: 25", "nombre: Ana", "edad: 30"
```

4.3.3 Tipo Especial: NULL

El valor `null` representa la ausencia explícita de valor. Una variable que no se ha definido todavía no debe confundirse con una variable definida cuyo valor es `null`.

Ejemplo:

• PHP

```
<?php
$variableSinValor = NULL;
$otraVariable = null; // Variable definida con valor null
?>
```

4.4 Conversión de Tipos en PHP

Aunque PHP gestiona automáticamente la mayoría de las conversiones de tipos (también llamado **coerción de tipos**), en algunos casos, es necesario realizar conversiones manuales para asegurarse de que una variable tiene el tipo de dato adecuado.

Ejemplo de conversión manual:

• PHP

```
<?php
    $numero = "25"; // Cadena de texto
    $numeroEntero = (int)$numero; // Conversión a entero
?>
```

Los tipos de conversión disponibles en PHP son:

- `(int)` o `(integer)` para convertir a entero.
- `(float)` o `(double)` o `(real)` para convertir a flotante.
- `(string)` para convertir a cadena de texto.
- `(bool)` o `(boolean)` para convertir a booleano.
- `(array)` para convertir a array.
- `(object)` para convertir a objeto.

4.5 Variables Predefinidas en PHP

PHP proporciona varias **variables superglobales**, accesibles en distintos ámbitos del script. Estas variables incluyen:

- `$_GET`: Almacena los parámetros pasados mediante la URL (método GET).
- `$_POST`: Almacena los datos enviados mediante formularios (método POST).
- `$_SESSION`: Almacena datos de la sesión de usuario.
- `$_COOKIE`: Almacena cookies.

Ejemplo de uso de `$_GET`:

• PHP

```
<?php
    // URL: http://example.com?nombre=Juan
    $nombre = $_GET['nombre'] ?? '';
    echo 'Hola, ' . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
?>
```

BLOQUE 04 · AMPLIACIÓN PARA EJERCICIOS

4.6 Arrays y operaciones utilizadas en las prácticas

Los ejercicios de la unidad incluyen tickets de compra, formularios con varias opciones, horarios y tests. Estas operaciones amplían las nociones de arrays del apartado 4.3.2.

Recorrer claves y valores con `foreach`

```
$productos = ["Pan" => 1.20, "Leche" => 1.50];
foreach ($productos as $producto => $precio) {
    echo "$producto: $precio €<br>";
}
```

En cada vuelta, `$producto` contiene la clave y `$precio` el valor. Si el formulario utiliza `name="aficiones[]"`, `$_POST['aficiones']` contendrá un array con los valores marcados, siempre que se haya seleccionado alguno.

Funciones para comprobar y seleccionar elementos

Función u operador	Uso en los ejercicios
<code>in_array(\$x, \$lista, true)</code>	Comprueba si un valor figura en una lista. El tercer argumento pide comparar también el tipo.
<code>array_key_exists(\$clave, \$datos)</code>	Comprueba si una clave está en un array asociativo.
<code>array_rand(\$datos, \$n)</code>	Elige aleatoriamente <i>claves</i> distintas. Con <code>\$n = 1</code> devuelve una sola clave; con <code>\$n > 1</code> devuelve un array de claves.
<code>count(\$lista)</code>	Devuelve el número de elementos de un array.
<code>unset(\$lista['clave'])</code>	Elimina un elemento por su clave.

```
$preguntas = ["Capital de España" => "Madrid",
              "2 + 2" => "4"];
$seleccion = (array) array_rand($preguntas, 1);
foreach ($seleccion as $clave) {
    echo $clave; // Se imprime una pregunta
}
```

Precaución: `isset($lista['clave'])` devuelve `false` si la clave existe pero almacena `null`; `array_key_exists()` distingue ese caso. No des por hecho que un dato enviado por un formulario sea una clave válida.

BLOQUE FORMATIVO

5. Operadores y Expresiones en PHP

En PHP, los **operadores** son símbolos especiales que se utilizan para realizar operaciones en variables y valores. Estos operadores permiten llevar a cabo diversas tareas como operaciones matemáticas, comparaciones entre valores, evaluaciones lógicas y más. Una **expresión** en PHP es cualquier combinación de valores, variables y operadores que devuelve un resultado. Comprender cómo usar operadores y expresiones es fundamental para manipular datos de manera eficaz dentro de un programa.

5.1 Operadores Aritméticos

Los **operadores aritméticos** en PHP permiten realizar operaciones matemáticas básicas como la suma, resta, multiplicación, división y el módulo. Estos operadores son utilizados comúnmente para manipular números y cálculos matemáticos dentro de un script.

Operador	Descripción	Ejemplo	Resultado
+	Suma	<code>\$a + \$b</code>	Suma de <code>\$a</code> y <code>\$b</code>
-	Resta	<code>\$a - \$b</code>	Resta de <code>\$b</code> a <code>\$a</code>
*	Multiplicación	<code>\$a * \$b</code>	Producto de <code>\$a</code> y <code>\$b</code>
/	División	<code>\$a / \$b</code>	Cociente de <code>\$a</code> entre <code>\$b</code>
%	Módulo (resto)	<code>\$a % \$b</code>	Resto de la división de <code>\$a</code> entre <code>\$b</code>

Ejemplo de operadores aritméticos en acción:

● PHP

```
<?php
$a = 10;
$b = 3;
echo $a + $b; // Imprime 13
echo $a - $b; // Imprime 7
echo $a * $b; // Imprime 30
echo $a / $b; // Imprime 3.3333...
echo $a % $b; // Imprime 1 (resto de 10 / 3)
?>
```

Precedencia de Operadores Aritméticos

La **precedencia** determina el orden en que se evalúan los operadores en una expresión. PHP sigue reglas de precedencia similares a las de matemáticas:

1. Las operaciones entre paréntesis `()` se evalúan primero.
2. **Multiplicación, división y módulo** tienen mayor precedencia que la **suma** y **resta**.

Ejemplo de precedencia de operadores:

• PHP

```
<?php
    $resultado = 5 + 3 * 2; // Resultado es 11, porque la multiplicación se evalúa primero.
    $resultado = (5 + 3) * 2; // Resultado es 16, porque la operación entre paréntesis se evalúa primero.
?>
```

5.2 Operadores de Asignación

Los operadores de asignación se utilizan para asignar valores a las variables. El operador más común es el de **asignación simple** (`=`), que asigna el valor de la expresión de la derecha a la variable de la izquierda. También existen **operadores de asignación combinados** que permiten realizar una operación aritmética y asignar el resultado en una sola expresión.

Operador	Descripción	Ejemplo	Equivalente
<code>=</code>	Asignación simple	<code>\$a = 5;</code>	Asigna el valor <code>5</code> a <code>\$a</code>
<code>+=</code>	Suma y asigna	<code>\$a += 3;</code>	<code>\$a = \$a + 3;</code>
<code>-=</code>	Resta y asigna	<code>\$a -= 3;</code>	<code>\$a = \$a - 3;</code>
<code>*=</code>	Multiplica y asigna	<code>\$a *= 3;</code>	<code>\$a = \$a * 3;</code>
<code>/=</code>	Divide y asigna	<code>\$a /= 3;</code>	<code>\$a = \$a / 3;</code>
<code>%=</code>	Módulo y asigna	<code>\$a %= 3;</code>	<code>\$a = \$a % 3;</code>

Ejemplo de operadores de asignación:

• PHP

```
<?php
    $a = 10;
    $a += 5; // Ahora $a es 15 (equivale a $a = $a + 5)
    $a *= 2; // Ahora $a es 30 (equivale a $a = $a * 2)
?>
```

5.3 Operadores de Comparación

Los operadores de comparación permiten comparar dos valores. El resultado de una comparación es siempre un valor booleano: **true** si la comparación es verdadera, o **false** si es falsa. Estos operadores son esenciales en **sentencias condicionales** y **bucles**.

Operador	Descripción	Ejemplo	Resultado
<code>==</code>	Igual a	<code>\$a == \$b</code>	True si <code>\$a</code> es igual a <code>\$b</code>
<code>===</code>	Idéntico (igual y mismo tipo)	<code>\$a === \$b</code>	True si <code>\$a</code> es igual a <code>\$b</code> y son del mismo tipo
<code>!=</code>	Diferente a	<code>\$a != \$b</code>	True si <code>\$a</code> no es igual a <code>\$b</code>
<code>!==</code>	No idéntico	<code>\$a !== \$b</code>	True si <code>\$a</code> no es igual a <code>\$b</code> o no son del mismo tipo

Operador	Descripción	Ejemplo	Resultado
>	Mayor que	<code>\$a > \$b</code>	True si <code>\$a</code> es mayor que <code>\$b</code>
<	Menor que	<code>\$a < \$b</code>	True si <code>\$a</code> es menor que <code>\$b</code>
>=	Mayor o igual que	<code>\$a >= \$b</code>	True si <code>\$a</code> es mayor o igual que <code>\$b</code>
<=	Menor o igual que	<code>\$a <= \$b</code>	True si <code>\$a</code> es menor o igual que <code>\$b</code>

Ejemplo de operadores de comparación:

• PHP

```
<?php
    $a = 10;
    $b = 20;

    var_dump($a == $b); // Imprime bool(false)
    var_dump($a != $b); // Imprime bool(true)
    var_dump($a < $b);  // Imprime bool(true)
?>
```

En este ejemplo, `var_dump()` imprime el valor booleano de la comparación. El resultado es `true` o `false` según si la expresión evaluada es verdadera o falsa.

5.4 Operadores Lógicos

Los operadores lógicos se utilizan para combinar múltiples condiciones o expresiones y devuelven un resultado booleano. Son esenciales para construir **condiciones complejas** en estructuras de control como `if` o `while`.

Operador	Descripción	Ejemplo	Resultado
&&	Y lógico (AND)	<code>\$a && \$b</code>	True si <code>\$a</code> y <code>\$b</code> son verdaderos
	O lógico (OR)	<code>\$a \$b</code>	True si <code>\$a</code> o <code>\$b</code> son verdaderos
!	Negación lógica (NOT)	<code>!\$a</code>	True si <code>\$a</code> es falso

Ejemplo de operadores lógicos:

• PHP

```
<?php
$a = true;
$b = false;

var_dump($a && $b); // Imprime bool(false) (ambas condiciones deben ser verdaderas)
var_dump($a || $b); // Imprime bool(true) (una de las dos condiciones es verdadera)
var_dump(!$a);      // Imprime bool(false) (negación de $a)

?>
```

- El operador `&&` (AND) devuelve **true** solo si ambas expresiones son verdaderas.
- El operador `||` (OR) devuelve **true** si al menos una de las expresiones es verdadera.
- El operador `!` (NOT) invierte el valor de una expresión. Si una expresión es **true**, la convierte en **false**, y viceversa.

5.5 Operadores de Incremento y Decremento

Estos operadores se utilizan para aumentar o disminuir el valor de una variable en **uno**. Pueden usarse en dos formas: **pre-incremento/decremento** o **post-incremento/decremento**.

Operador	Descripción	Ejemplo	Resultado
<code>++\$a</code>	Pre-incremento	<code>\$a = 5; ++\$a;</code>	<code>\$a</code> ahora es 6
<code>\$a++</code>	Post-incremento	<code>\$a = 5; \$a++;</code>	La expresión devuelve 5 y después <code>\$a</code> pasa a valer 6
<code>--\$a</code>	Pre-decremento	<code>\$a = 5; --\$a;</code>	<code>\$a</code> ahora es 4
<code>\$a--</code>	Post-decremento	<code>\$a = 5; \$a--;</code>	La expresión devuelve 5 y después <code>\$a</code> pasa a valer 4

Ejemplo de incremento/decremento:

• PHP

```
<?php
$a = 5;
echo ++$a; // Imprime 6 (pre-incremento)
echo $a++; // Imprime 6 (pero después $a será 7)
echo $a;   // Imprime 7

?>
```

BLOQUE FORMATIVO

6. Estructuras de Control en PHP

Las **estructuras de control** en PHP permiten modificar el flujo de ejecución de un programa en función de ciertas condiciones o repetir bloques de código. Estas estructuras son esenciales para crear programas dinámicos y flexibles, ya que permiten ejecutar diferentes secciones de código basándose en entradas o valores variables. En PHP, las principales estructuras de control incluyen las **condicionales** (`if`, `else`, `elseif`, `switch`) y los **bucles** (`for`, `while`, `foreach`).

6.1 Estructuras Condicionales

Las estructuras condicionales permiten ejecutar bloques de código en función de si una condición es verdadera o falsa. PHP ofrece varias formas de manejar condiciones, siendo las más comunes `if`, `else`, `elseif` y `switch`.

6.1.1 Condicional `if`, `else`, `elseif`

La estructura condicional `if` evalúa una expresión y ejecuta un bloque de código si la expresión es verdadera. Si la expresión es falsa, puede haber una rama `else` para ejecutar un bloque de código alternativo. Además, se pueden encadenar múltiples condiciones utilizando `elseif`.

Sintaxis básica:

- PHP

```
if (condición) {  
    // Bloque de código si la condición es verdadera  
} elseif (otra_condición) {  
    // Bloque de código si la segunda condición es verdadera  
} else {  
    // Bloque de código si ninguna condición es verdadera  
}
```

Ejemplo de uso de `if`, `else`, `elseif`:

- PHP

```
<?php  
$edad = 20;  
  
if ($edad >= 18) {  
    echo "Eres mayor de edad."; // Se ejecuta si $edad es mayor o igual a 18  
} else {  
    echo "Eres menor de edad."; // Se ejecuta si $edad es menor que 18  
}  
?>
```

En este ejemplo, se evalúa si la variable `$edad` es mayor o igual a 18. Dependiendo del resultado, se ejecuta uno de los dos bloques de código.

Uso de `elseif`:

• PHP

```
<?php
    $nota = 75;

    if ($nota >= 90) {
        echo "Excelente";
    } elseif ($nota >= 75) {
        echo "Muy bien";
    } else {
        echo "Necesitas mejorar";
    }
?>
```

Aquí, el bloque `elseif` permite evaluar otra condición en caso de que la primera no sea verdadera.

6.2 Estructuras de Repetición (Bucles)

Los **bucles** permiten ejecutar un bloque de código repetidamente, ya sea un número fijo de veces o hasta que una condición se cumpla. PHP proporciona varias estructuras de bucles que se adaptan a diferentes situaciones: `for`, `while`, y `foreach`.

6.2.1 Bucle `for`

El bucle `for` se utiliza cuando el número de iteraciones es conocido de antemano. Se compone de tres partes principales:

1. Inicialización de la variable de control.
2. Condición para continuar el bucle.
3. Actualización de la variable de control al final de cada iteración.

Sintaxis básica de `for`:

• PHP

```
for (inicialización; condición; actualización) {
    // Bloque de código a repetir
}
```

Ejemplo de bucle `for`:

• PHP

```
<?php
    for ($i = 0; $i < 5; $i++) {
        echo "Iteración número: $i<br>";
    }
?>
```

En este ejemplo, el bucle comienza con `$i = 0`, y mientras `$i` sea menor que 5, el bucle se repite. Al final de cada iteración, `$i` se incrementa en 1.

6.2.2 Bucle `while`

El bucle `while` ejecuta un bloque de código repetidamente mientras una condición sea verdadera. A diferencia del `for`, la condición se evalúa antes de cada iteración, lo que lo hace útil cuando no se sabe cuántas veces se repetirá el bucle.

Sintaxis básica de `while` :

- PHP

```
while (condición) {  
    // Bloque de código a repetir mientras la condición sea verdadera  
}
```

Ejemplo de bucle `while` :

- PHP

```
<?php  
    $i = 0;  
  
    while ($i < 5) {  
        echo "Iteración número: $i<br>";  
        $i++; // Incrementar $i para evitar un bucle infinito  
    }  
?  
>
```

En este ejemplo, el bucle se ejecuta mientras `$i` sea menor que 5. En cada iteración, se imprime el valor de `$i` y luego se incrementa.

6.2.3 Bucle `foreach`

El bucle `foreach` está diseñado específicamente para recorrer **arrays**. En cada iteración, el valor de un elemento del array se asigna a una variable temporal, lo que permite procesar cada elemento del array fácilmente.

Sintaxis básica de `foreach` :

- PHP

```
foreach (array as valor) {  
    // Bloque de código a repetir para cada elemento del array  
}
```

Ejemplo de bucle `foreach` :

- PHP

```
<?php  
    $frutas = array("manzana", "naranja", "pera");  
  
    foreach ($frutas as $fruta) {  
        echo "Fruta: $fruta<br>";  
    }  
?  
>
```

En este ejemplo, el bucle `foreach` recorre el array `$frutas` y en cada iteración, asigna el valor del elemento actual a la variable `$fruta`, que luego se imprime. El bucle continúa hasta que se han procesado todos los elementos del array.

Bucle `foreach` con claves y valores:

Si el array es asociativo (map), el bucle `foreach` puede recorrer tanto las **claves** como los **valores**.

Ejemplo:

• PHP

```
<?php
$persona = array("nombre" => "Juan", "edad" => 25);

foreach ($persona as $clave => $valor) {
    echo "$clave: $valor<br>";
}

?>
```

Aquí, `$clave` toma los nombres de las claves (`nombre`, `edad`), y `$valor` toma los valores asociados (`Juan`, `25`).

6.3 Otras Estructuras de Control

6.3.1 Bucle `do...while`

El bucle `do...while` es similar al bucle `while`, pero la diferencia clave es que **siempre** se ejecuta al menos una vez, ya que la condición se evalúa **después** de ejecutar el bloque de código.

Sintaxis básica de `do...while`:

• PHP

```
do {
    // Bloque de código a ejecutar
} while (condición);
```

Ejemplo de `do...while`:

• PHP

```
<?php
$i = 0;

do {
    echo "Iteración número: $i<br>";
    $i++;
} while ($i < 5);

?>
```

En este ejemplo, el bloque de código dentro de `do` se ejecuta al menos una vez, incluso si la condición no se cumple desde el inicio.

BLOQUE 06 · AMPLIACIÓN PARA EJERCICIOS

6.4 Funciones y reutilización de código

En las prácticas del IMC, matrícula, concesionario y test se utilizan **funciones propias** para no repetir instrucciones. Una función recibe datos mediante *parámetros*, ejecuta un bloque y puede devolver un valor con `return`.

```
function calcularSubtotal($precio, $cantidad) {  
    return $precio * $cantidad;  
}  
$total = calcularSubtotal(12.5, 3);  
echo $total; // 37.5
```

Al llegar a `return`, termina la ejecución de la función y se devuelve el resultado a la expresión que la llamó. Los parámetros `$precio` y `$cantidad` pertenecen al ámbito local de la función.

Ámbito de variables y `global`

Por defecto, una variable definida fuera de la función no está disponible automáticamente dentro. En los ejemplos proporcionados aparece `global`, que permite acceder a una variable del ámbito global. Para nuevas soluciones es preferible pasar la información necesaria como parámetro.

```
$tarifa = 10;  
function calcularPrecio($unidades, $tarifa) {  
    return $unidades * $tarifa;  
}  
echo calcularPrecio(3, $tarifa); // 30
```

Separar archivos con `require_once`

La instrucción `require_once` carga otro archivo PHP y evita cargarlo de nuevo si ya estaba incluido. Es útil para almacenar en un archivo los arrays de precios, asignaturas o preguntas y reutilizarlos desde un procesador.

```
// En componentes.php se define $componentes.  
require_once __DIR__ . '/componentes.php';  
echo $componentes['Modelo']['SUV'];
```

`__DIR__` representa el directorio del archivo actual. Así, una ruta relativa a ese directorio funciona aunque cambie el directorio desde el que se llame al programa.

Idea clave: el archivo importado también se ejecuta en el servidor. El navegador recibe la respuesta generada, no el contenido del archivo PHP.

BLOQUE 06 · AMPLIACIÓN PARA EJERCICIOS

6.5 Funciones auxiliares: cadenas, fechas y cálculos

Las prácticas aplican funciones incorporadas para convertir horas, comprobar textos y mostrar cantidades.

Horas semanales con `strtotime`

`strtotime()` intenta convertir una fecha u hora textual en una marca de tiempo. Para dos horas del mismo día, la diferencia en segundos, dividida entre 3600, expresa la duración en horas.

```
$inicio = strtotime('08:30');
$fin = strtotime('10:00');
$horas = ($fin - $inicio) / 3600;
echo $horas; // 1.5
```

Este ejemplo presupone dos horas válidas de un mismo día y que la hora final es posterior a la inicial. Para horarios que cruzan la medianoche habría que tratar el cambio de día.

Reemplazar texto y preparar una salida

```
$codigo = str_replace('-', ' ', '12-34');
echo $codigo; // 1234

$precio = round(12.345, 2); // 12.35
$importe = number_format($precio, 2, ',', '.');
echo $importe; // 12,35
```

Expresión	Para qué sirve
<code>str_replace(\$buscar, \$nuevo, \$texto)</code>	Sustituir una subcadena por otra.
<code>round(\$numero, 2)</code>	Redondear un número a dos decimales.
<code>number_format(...)</code>	Mostrar un número como texto con separadores.
<code>htmlspecialchars(\$texto, ENT_QUOTES, 'UTF-8')</code>	Escapar caracteres especiales antes de insertar datos externos en texto HTML.

Sobre el IVA en los ejercicios: si un precio es una *base sin impuestos*, $\text{IVA} = \text{base} * 0.21$ y $\text{total} = \text{base} + \text{IVA}$. Si el precio *ya incluye IVA*, la parte de IVA contenida en él es $\text{total} - \text{total} / 1.21$. Son supuestos distintos; elige uno y mantenlo coherente.

BLOQUE 06 · AMPLIACIÓN PARA EJERCICIOS

6.6 Comprobaciones y funciones de apoyo

En los archivos de referencia se usan además las siguientes herramientas para validar, transformar y presentar los datos. Todas se ejecutan en el servidor.

Trabajar con texto recibido del formulario

```
$nombre = trim(' Ana '); // 'Ana'
$codigo = strtoupper('carSale10'); // 'CARSALE10'
$signatura = ucfirst('matematicas'); // 'Matematicas'
```

Función o construcción	Qué hace
<code>trim(\$texto)</code>	Elimina espacios al principio y al final.
<code>strtoupper(\$texto) / ucfirst(\$texto)</code>	Convierte a mayúsculas una cadena / su primer carácter (según configuración y caracteres).
<code>preg_match_all('/./us', \$texto)</code>	En estos ejemplos cuenta caracteres Unicode de una cadena; devuelve el número de coincidencias.
<code>is_string(\$dato) / is_array(\$dato)</code>	Comprueba si un valor es texto o un array.
<code>array_unique(\$lista)</code>	Elimina elementos repetidos de un array; conserva las claves existentes.

Datos ausentes, comparación estricta y errores

```
$aficiones = $_POST['aficiones'] ?? [];
if (!is_array($aficiones)) {
    exit('Selección no válida.');
```

```
}
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Este archivo requiere POST.');
```

```
}
```

El operador `??` usa un valor alternativo si la variable no existe o contiene `null`. El operador `===` exige igualdad de valor y tipo; `!==` significa lo contrario. `exit()` detiene el script; `http_response_code(405)` establece un código de estado HTTP que indica que el método no está permitido.

Regla de validación: antes de acceder a una clave recibida, comprueba que el dato existe, tiene el tipo esperado y pertenece al conjunto de valores permitido. Así evitas avisos, errores y datos manipulados.

BLOQUE FORMATIVO

7. Trabajo con Formularios en PHP

Los **formularios HTML** son una de las formas más comunes para que los usuarios interactúen con un sitio web. A través de ellos, los usuarios pueden enviar información al servidor, que es procesada por un script en el servidor. En PHP, el procesamiento de formularios se maneja principalmente mediante los métodos **GET** y **POST**, que determinan cómo se envían los datos al servidor.

7.1 Métodos de Envío de Formularios: GET vs POST

Existen dos métodos principales para enviar datos de formularios HTML al servidor: **GET** y **POST**. Cada uno tiene características y usos específicos que los hacen más apropiados según el contexto.

7.1.1 Método GET

El método **GET** envía los datos del formulario a través de la **URL**. Los datos se adjuntan como una **cadena de consulta** (query string), lo que significa que son visibles en la barra de direcciones del navegador. Resulta conveniente para búsquedas y otras solicitudes que no modifican datos, pero **no es adecuado para enviar contraseñas u otros datos sensibles**, pues pueden quedar registrados en el historial y otros registros de URL. Los límites de longitud de la URL dependen del cliente y del servidor.

Características del método GET:

- Los datos enviados son visibles en la barra de direcciones del navegador.
- No es adecuado para enviar datos confidenciales (como contraseñas o información personal).
- Hay un límite en la cantidad de datos que se pueden enviar, ya que las URL tienen una longitud máxima.
- Se puede almacenar en caché y reutilizar, lo que lo hace útil para solicitudes idempotentes (que no modifican el estado del servidor).

Ejemplo de un formulario que usa el método GET:

• HTML

```
<form method="GET" action="procesar.php">
  Nombre: <input type="text" name="nombre">
  <input type="submit" value="Enviar">
</form>
```

En este caso, al enviar el formulario, los datos se añaden a la URL, por ejemplo:

```
http://example.com/procesar.php?nombre=Juan
```

7.1.2 Método POST

El método **POST** envía los datos en el **cuerpo de la solicitud HTTP**, en lugar de añadirlos a la URL. Esto es útil para formularios que crean o modifican información, y permite enviar cuerpos de mayor tamaño que una URL. Los datos no aparecen en la barra de direcciones, pero **POST no cifra la información ni garantiza su confidencialidad**: utiliza **HTTPS** para protegerla durante el transporte. El servidor puede imponer límites al tamaño de la solicitud.

Características del método POST:

- Los datos no son visibles en la barra de direcciones; aun así, emplea HTTPS cuando deban viajar protegidos.
- El cuerpo puede tener límites establecidos por el servidor y por la configuración de PHP.
- Se utiliza habitualmente para enviar formularios que **modifican** el estado del servidor, como registros o compras.
- No constituye por sí mismo una medida de seguridad frente a XSS, CSRF o inyección SQL.

Ejemplo de un formulario que usa el método POST:

• HTML

```
<form method="POST" action="procesar.php">
  Nombre: <input type="text" name="nombre">
  <input type="submit" value="Enviar">
</form>
```

7.2 Procesamiento de Formularios en PHP

Cuando un formulario se envía desde el navegador al servidor, PHP puede acceder a los datos enviados a través de **variables superglobales** como `$_GET` y `$_POST`, dependiendo del método de envío utilizado.

7.2.1 Superglobales: `$_GET` y `$_POST`

- **`$_GET`**: Es una superglobal en PHP que contiene todos los datos enviados por el método GET. Los valores enviados se almacenan en un array asociativo donde las claves corresponden a los nombres de los campos del formulario.

Ejemplo de acceso a datos enviados por GET:

• PHP

```
<?php
  $nombre = $_GET["nombre"] ?? '';
  echo 'Hola, ' . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
?>
```

- **`$_POST`**: Similar a `$_GET`, pero se utiliza para capturar los datos enviados por el método POST. Los datos se almacenan en un array asociativo y pueden ser recuperados usando los nombres de los campos de formulario.

Ejemplo de acceso a datos enviados por POST:

BLOQUE 07 · AMPLIACIÓN PARA EJERCICIOS

7.1.3 Formularios con varias opciones

En las prácticas recibirás datos de diferentes controles HTML. Todos se envían mediante el atributo `name` del control; el atributo `id` sirve para asociarlo con una etiqueta `label`.

Control HTML	Cómo llega a PHP
<code>input type="text", number</code>	Cadena en <code>\$_POST['nombre']</code> o <code>\$_GET['nombre']</code> .
<code>input type="radio"</code>	Solo se envía la opción elegida de un grupo con el mismo <code>name</code> .
<code>input type="checkbox"</code>	Los no marcados no se envían. Con <code>name="aficiones[]"</code> se obtiene un array de valores marcados.
<code>select</code>	Envía el <code>value</code> de la opción elegida.
<code>input type="hidden"</code>	Envía un dato no visible en el formulario, pero modificable por el usuario.

```
<form action="procesar.php" method="post">
  <label><input type="checkbox" name="aficiones[]"
    value="cine"> Cine</label>
  <label><input type="checkbox" name="aficiones[]"
    value="musica"> Música</label>
  <button type="submit">Enviar</button>
</form>
```

```
<?php
$aficiones = $_POST['aficiones'] ?? [];
if (!is_array($aficiones)) { $aficiones = []; }
foreach ($aficiones as $aficion) {
    echo htmlspecialchars($aficion, ENT_QUOTES, 'UTF-8');
    echo '<br>';
}
?>
```

Regla práctica: no confíes en `required`, `min` o `max` como única validación: actúan en el navegador, mientras que PHP debe comprobar los datos recibidos. Tampoco asumas que un checkbox existe en `$_POST` si no se marcó.

BLOQUE 07 · AMPLIACIÓN PARA EJERCICIOS

7.1.4 Etiquetas HTML que aparecen en las prácticas

En los ejemplos se combinan **HTML** para recoger datos y **PHP** para procesarlos. El nombre del archivo indicado en **action** debe coincidir con el del procesador que se encuentra en la misma carpeta.

Controles habituales

Etiqueta / atributo	Función
<code><form action="..." method="post"></code>	Indica a qué archivo y con qué método HTTP se envía el formulario.
<code><label for="id"></code>	Asocia un texto descriptivo con el control cuyo id coincide.
<code><fieldset></code> / <code><legend></code>	Agrupar controles relacionados y presenta el título del grupo.
<code><select name="edad"></code> y <code><option value="..."></code>	Permiten elegir un valor de un menú desplegable.
<code>required</code> , <code>min</code> , <code>max</code> , <code>step</code>	Ayudan al navegador a comprobar presencia, rango y paso de números; PHP debe validar igualmente.
<code>type="submit"</code> / <code>type="reset"</code>	Enviar los datos / restaurar los valores iniciales del formulario.
<code>name="opciones[]"</code>	Envía varias casillas marcadas con la misma clave como array.

Ejemplo completo: radio, casillas y campo oculto

```
<form action="procesar.php" method="post">
  <fieldset>
    <legend>Preferencias</legend>
    <label><input type="radio" name="turno"
      value="manana" required> Mañana</label>
    <label><input type="radio" name="turno"
      value="tarde"> Tarde</label>
    <label><input type="checkbox" name="modulos[]"
      value="DWES"> DWES</label>
  </fieldset>
  <input type="hidden" name="origen" value="ejercicio">
  <button type="submit">Enviar</button>
</form>
```

En la tabla del horario, las reglas CSS con **class** asignan colores a las celdas según la asignatura. Al procesarlo, `$_POST['turno']` guarda una opción, `$_POST['modulos']` ?? [] puede contener varias y `$_POST['origen']` incluye el valor oculto. **Oculto no significa seguro:** el servidor debe validar los tres datos antes de utilizarlos.

• PHP

```
<?php
    $nombre = $_POST["nombre"] ?? '';
    echo 'Hola, ' . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
?>
```

7.3 Validación de Formularios en PHP: Un Enfoque Integral

En el desarrollo web, la **validación de formularios** es un proceso crítico que garantiza la integridad, seguridad y precisión de los datos recibidos. Cuando los usuarios interactúan con formularios, los datos que envían pueden estar incompletos, incorrectos o maliciosos, lo que podría afectar la seguridad y funcionalidad de la aplicación. Por ello, resulta fundamental implementar mecanismos robustos de validación tanto del lado del cliente como del lado del servidor.

La validación de formularios en PHP se puede abordar desde dos enfoques complementarios: **validación del lado del cliente** (normalmente realizada con JavaScript) y **validación del lado del servidor** (llevada a cabo con PHP). Aunque la validación del lado del cliente mejora la experiencia del usuario al proporcionar retroalimentación inmediata, **no debe ser considerada suficiente**. Dado que el código JavaScript del lado del cliente puede ser modificado o deshabilitado, es imperativo que la validación **siempre se realice también del lado del servidor**, garantizando así la protección contra posibles entradas maliciosas o errores que podrían comprometer el sistema.

7.3.1 Validación Básica del Lado del Servidor: Seguridad y Precisión

La validación del lado del servidor se refiere a la verificación y limpieza de los datos recibidos antes de procesarlos en el backend. En este contexto, una práctica básica pero esencial es asegurarse de que los campos obligatorios no estén vacíos, así como validar que los datos ingresados sean del tipo esperado. Esto previene la introducción de datos incorrectos o inconsistentes que podrían generar errores o vulnerabilidades en la aplicación.

A continuación, se presenta un ejemplo típico de validación básica en PHP. El objetivo es comprobar si el campo `nombre` ha sido completado por el usuario, mostrando un mensaje de error si el campo está vacío o procesando el dato en caso contrario.

• PHP

```
<?php
    if ($_SERVER["REQUEST_METHOD"] === "POST") {
        if (trim($_POST["nombre"]) ?? '' === '') {
            echo "El nombre es obligatorio.";
        } else {
            $nombre = trim($_POST["nombre"] ?? '');
            echo 'Hola, ' . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
        }
    }
?>
```

En este ejemplo, el script primero verifica si el formulario fue enviado utilizando la superglobal `$_SERVER["REQUEST_METHOD"]`, que determina si la solicitud HTTP es de tipo `POST`. Luego, mediante la función `empty()`, se valida que el campo `nombre` no esté vacío. Si lo está, se notifica al usuario con un mensaje de error. Si el campo contiene un valor, este se procesa y se muestra como parte de la respuesta.

7.3.2 Saneamiento y Validación de Datos: Protección Contra Amenazas de Seguridad

Los datos recibidos de un formulario no son confiables por defecto. Deben **validarse** según los requisitos del campo y **tratarse según su destino**: usa consultas preparadas para SQL, escapa los valores al mostrarlos en HTML y evita ejecutar entradas del usuario como comandos. No existe una única operación de saneamiento que proteja frente a todos los riesgos.

Uso de `htmlspecialchars()` al mostrar texto en HTML

Una función comúnmente empleada para este propósito es `htmlspecialchars()`, que convierte caracteres especiales en entidades HTML, evitando así que los usuarios puedan inyectar código HTML o scripts maliciosos.

• PHP

```
<?php
    if ($_SERVER["REQUEST_METHOD"] === "POST") {
        $nombre = $_POST["nombre"] ?? '';
        echo 'Hola, ' . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
    }
?>
```

En este ejemplo, `htmlspecialchars()` transforma caracteres con significado especial en HTML para que el nombre se muestre como texto. Este escape es adecuado al insertar texto en HTML, pero no sustituye a la validación ni protege por sí solo otros contextos, como una URL, JavaScript o una consulta SQL.

Validación de Tipos de Datos con `filter_var()`

Además del saneamiento, la validación estricta de los tipos de datos es crucial para asegurar que la información ingresada sea la esperada. PHP proporciona una serie de funciones, entre ellas `filter_var()`, que permiten validar diferentes tipos de datos, como correos electrónicos, URLs y números enteros. La función `filter_var()` no solo valida, sino que también puede sanitizar los datos utilizando diferentes filtros.

Por ejemplo, para validar que el dato ingresado en un campo de correo electrónico sea un formato válido, se puede usar el siguiente código:

• PHP

```
<?php
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        $email = $_POST["email"];
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            echo "Correo electrónico no válido.";
        } else {
            echo "Correo válido: " . htmlspecialchars($email, ENT_QUOTES, "UTF-8");
        }
    }
?>
```

En este fragmento de código, la función `filter_var()` utiliza el filtro `FILTER_VALIDATE_EMAIL` para comprobar si el valor ingresado cumple con el formato estándar de un correo electrónico. Si el valor no es válido, se informa al usuario; de lo contrario, el correo electrónico es aceptado y procesado.

7.3.3 Consideraciones Adicionales: Mejores Prácticas para la Validación en PHP con Ejemplos

La validación y saneamiento de datos no debe limitarse a ejemplos sencillos como verificar campos vacíos o validar correos electrónicos. Cada aplicación tiene requisitos específicos y, por tanto, la validación debe adaptarse a la naturaleza de los datos que se procesan. A continuación se presentan mejores prácticas adicionales para la validación en PHP, junto con ejemplos prácticos que ilustran cómo implementarlas.

Validación de Campos Numéricos y Rangos

Cuando se trabaja con datos numéricos, es crucial asegurarse de que el valor recibido es, efectivamente, un número válido y que se encuentra dentro de los rangos definidos por el sistema. En PHP, se puede utilizar la función `filter_var()` junto con los filtros `FILTER_VALIDATE_INT` o `FILTER_VALIDATE_FLOAT` para validar números enteros o decimales, respectivamente.

Ejemplo: Validación de Edad con Rango Permitido

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $edad = $_POST["edad"];
    // Validar que es un número entero y que está entre 18 y 65 años
    if (filter_var($edad, FILTER_VALIDATE_INT, array("options" => array("min_range" => 18, "max_range" => 65))) === false) {
        echo "La edad debe ser un número entero entre 18 y 65.";
    } else {
        echo "Edad válida: $edad";
    }
}
?>
```

En este ejemplo, `filter_var()` asegura que el campo `edad` es un número entero y está dentro del rango de 18 a 65 años. Si no cumple con estos requisitos, se devuelve un mensaje de error. De lo contrario, se procesa el dato.

Ejemplo: Validación de Precio Decimal

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $precio = $_POST["precio"];
    // Validar que es un número decimal con filtro y un rango de precio
    $precioValidado = filter_var($precio, FILTER_VALIDATE_FLOAT);
    if ($precioValidado === false || $precioValidado < 0.01 || $precioValidado > 10000) {
        echo "El precio debe ser un número decimal entre 0.01 y 10,000.";
    } else {
        echo "Precio válido: $precio";
    }
}
?>
```

Aquí se valida que el campo `precio` es un número decimal, y que está dentro del rango definido, asegurando que no se acepten valores negativos o extremadamente altos.

Validación de Longitud de Campos

En muchos casos, es necesario limitar la longitud de los datos ingresados, especialmente cuando se trata de campos como nombres, contraseñas o comentarios. Esto previene errores de almacenamiento y protege contra posibles ataques que intentan desbordar buffers o bases de datos. Se puede utilizar `mb_strlen()` para medir la longitud de las cadenas de texto en caracteres Unicode, siempre que esté instalada la extensión `mbstring`. `strlen()` cuenta bytes, por lo que puede dar un resultado distinto con letras acentuadas.

Ejemplo: Validación de Longitud del Nombre

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre = $_POST["nombre"] ?? '';
    // Validar que la longitud del nombre esté entre 3 y 50 caracteres
    if (mb_strlen($nombre, "UTF-8") < 3 || mb_strlen($nombre, "UTF-8") > 50) {
        echo "El nombre debe tener entre 3 y 50 caracteres.";
    } else {
        echo "Nombre válido: " . htmlspecialchars($nombre, ENT_QUOTES, "UTF-8");
    }
}
?>
```

En este caso, se valida que el campo `nombre` tenga entre 3 y 50 caracteres, evitando que se ingresen nombres demasiado cortos o excesivamente largos.

Ejemplo: Validación de Contraseña con Longitud y Complejidad

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $password = $_POST["password"];
    // Validar que la contraseña tenga al menos 8 caracteres y contenga letras y números
    if (strlen($password) < 8 || !preg_match("/[A-Za-z]/", $password) || !preg_match("/[0-9]/", $password)) {
        echo "La contraseña debe tener al menos 8 caracteres, incluir letras y números.";
    } else {
        echo "Contraseña válida.";
    }
}
?>
```

Este código valida que la contraseña tenga una longitud mínima de 8 caracteres, y que incluya tanto letras como números, mejorando la seguridad del sistema.

Validación de Entradas Personalizadas con Expresiones Regulares

En algunos casos, la validación puede requerir patrones más específicos, como el formato de un número de teléfono, código postal o incluso nombres de usuario. Las **expresiones regulares** (regular expressions o **regex**) son una poderosa herramienta para definir estos patrones y asegurar que los datos ingresados cumplen con las expectativas.

Ejemplo: Validación de Número de Teléfono

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $telefono = $_POST["telefono"];
    // Validar que el número de teléfono sea de 10 dígitos
    if (!preg_match("/^[0-9]{10}$/", $telefono)) {
        echo "El número de teléfono debe contener exactamente 10 dígitos.";
    } else {
        echo "Número de teléfono válido.";
    }
}
?>
```

Aquí se utiliza una expresión regular que asegura que el campo `telefono` contiene exactamente 10 dígitos, un formato típico para números telefónicos en muchos países.

Ejemplo: Validación de Código Postal

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $codigo_postal = $_POST["codigo_postal"];
    // Validar que el código postal sea un formato válido (5 dígitos en este caso)
    if (!preg_match("/^[0-9]{5}$/", $codigo_postal)) {
        echo "El código postal debe contener 5 dígitos.";
    } else {
        echo "Código postal válido.";
    }
}
?>
```

En este caso, se valida que el código postal ingresado tenga exactamente 5 dígitos numéricos, un formato estándar en muchos países.

Ejemplo: Validación de Nombre de Usuario

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"];
    // Validar que el nombre de usuario contiene solo letras, números y guiones bajos, y tiene
    // entre 3 y 15 caracteres
    if (!preg_match("/^[a-zA-Z0-9_]{3,15}$/", $username)) {
        echo "El nombre de usuario debe tener entre 3 y 15 caracteres y solo puede contener
        letras, números y guiones bajos.";
    } else {
        echo "Nombre de usuario válido: " . htmlspecialchars($username, ENT_QUOTES, "UTF-8");
    }
}
?>
```

Este ejemplo muestra cómo validar un nombre de usuario que debe contener solo letras, números y guiones bajos, y tener una longitud entre 3 y 15 caracteres.

Validación de Fechas

Las fechas son un tipo de dato especial que también requiere validación. PHP proporciona la función `checkdate()` para validar fechas con formato día, mes y año.

Ejemplo: Validación de Fecha Válida

• PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $dia = $_POST["dia"];
    $mes = $_POST["mes"];
    $anio = $_POST["anio"];
    // Validar si la fecha es válida
    if (!checkdate($mes, $dia, $anio)) {
        echo "La fecha no es válida.";
    } else {
        echo "Fecha válida: $dia/$mes/$anio";
    }
}
?>
```

En este ejemplo, se valida que la fecha ingresada sea válida utilizando `checkdate()`, que comprueba que el día, mes y año forman una fecha real.

7.4 Subida de Archivos en Formularios

PHP también permite el manejo de **subida de archivos** a través de formularios. Para habilitar esta funcionalidad, es necesario agregar el atributo `enctype="multipart/form-data"` en el formulario y usar la superglobal `$_FILES` para acceder a los archivos subidos.

Ejemplo de formulario para subir archivos:

• HTML

```
<form method="POST" action="upload.php" enctype="multipart/form-data">
    Selecciona un archivo: <input type="file" name="archivo">
    <input type="submit" value="Subir">
</form>
```

Procesamiento en PHP para subir imágenes (`upload.php`):

El siguiente ejemplo acepta imágenes JPEG y PNG de hasta 2 MB. Se deben crear previamente la carpeta de destino y sus permisos, y evitar que el servidor ejecute como código los archivos guardados en ella.

• PHP

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $archivo = $_FILES['archivo'] ?? null;

    if ($archivo === null || $archivo['error'] !== UPLOAD_ERR_OK) {
        exit('No se ha recibido un archivo válido.');
```

```
    }

    if ($archivo['size'] > 2 * 1024 * 1024) {
        exit('La imagen no puede superar los 2 MB.');
```

```
    }

    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($archivo['tmp_name']);
    $extensiones = ['image/jpeg' => 'jpg', 'image/png' => 'png'];
    if (!isset($extensiones[$mime])) {
        exit('Solo se admiten imágenes JPEG y PNG.');
```

```
    }

    $nombreSeguro = bin2hex(random_bytes(16)) . '.' . $extensiones[$mime];
    $destino = __DIR__ . '/uploads/' . $nombreSeguro;

    if (move_uploaded_file($archivo['tmp_name'], $destino)) {
        echo 'Imagen recibida correctamente.';
    } else {
        echo 'No se pudo guardar la imagen.';
    }
}
```

`move_uploaded_file()` mueve un archivo recibido mediante HTTP a una ruta del servidor. En este ejemplo se comprueba el estado de la subida, se limita el tamaño, se valida el tipo detectado y se genera un nombre nuevo para evitar usar directamente el nombre enviado por el usuario. En una aplicación real también deben configurarse los permisos del directorio de subida y las medidas de almacenamiento apropiadas.

BLOQUE 07 · AMPLIACIÓN PARA EJERCICIOS

7.5 Validar formularios de ejercicios completos

Para procesar matrículas, coches o tests, PHP debe comprobar **método HTTP, presencia, tipo, rango y valores permitidos**. Un formulario bien formado ayuda al usuario, pero no sustituye la validación del lado del servidor.

1. Comprobar método y leer datos sin avisos

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Utiliza el formulario para enviar los datos.');
```

2. Validar enteros y opciones de una lista

```
$cantidad = filter_var($_POST['cantidad'] ?? null,
    FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 1, 'max_range' => 5]]);
if ($cantidad === false) {
    exit('La cantidad debe estar entre 1 y 5.');
```

3. Validar arrays de casillas y mostrar su contenido

```
$seleccion = $_POST['asignaturas'] ?? [];
if (!is_array($seleccion)) {
    exit('Las asignaturas deben enviarse como lista.');
```

Al mostrar datos enviados por el usuario dentro de HTML, aplica `htmlspecialchars()`. Si el dato se inserta en una consulta SQL, utiliza consultas preparadas: el escape de HTML no protege una consulta SQL.

BLOQUE 07 · AMPLIACIÓN PARA EJERCICIOS

7.6 Tests aleatorios: selección y corrección

El generador de tests combina arrays asociativos, selección aleatoria de preguntas, campos ocultos y un procesador que contabiliza los aciertos. Estos son los pasos relevantes del ejercicio avanzado.

Elegir preguntas distintas al azar

```
$repositorio = [  
    '¿Cuánto es 2 + 2?' => ['3', '4', '5'],  
    '¿Cuánto es 3 + 3?' => ['5', '6', '7']  
];  
$cantidad = 1;  
$claves = (array) array_rand($repositorio, $cantidad);  
foreach ($claves as $pregunta) {  
    echo htmlspecialchars($pregunta, ENT_QUOTES, 'UTF-8');  
}
```

`array_rand()` devuelve las **claves**, no las respuestas. Convertir el resultado a `array` permite iterarlo incluso si solo se ha elegido una pregunta.

Corregir también las preguntas sin respuesta

Un botón de radio no seleccionado no llega en el envío. Para obtener una nota justa, el programa debe conocer la lista de preguntas mostradas, contar todas y sumar un acierto solo si la respuesta recibida coincide con la solución del repositorio.

```
$preguntas = $_POST['preguntas'] ?? [];  
$respuestas = $_POST['respuestas'] ?? [];  
if (!is_array($preguntas) || count($preguntas) === 0) {  
    exit('No se ha recibido ningún test válido.');
```

```
}
```

```
$aciertos = 0;
```

```
foreach ($preguntas as $i => $pregunta) {
```

```
    $respuesta = $respuestas[$i] ?? '';
```

```
    if (($soluciones[$pregunta] ?? null) === $respuesta) {
```

```
        $aciertos++;
```

```
    }
```

```
}
```

```
$nota = round($aciertos / count($preguntas) * 10, 2);
```

Importante: los campos ocultos y las opciones del formulario se pueden modificar desde el navegador. El corrector debe contrastar las preguntas y respuestas recibidas con el repositorio del servidor antes de calcular una nota. Este ejemplo muestra la idea básica; el ejercicio completo incorpora esas comprobaciones.